Career Scout Profile Agent

Architecture and Operations Manual

November 5 2026

This manual brings together the Profile Agent documentation, agent interaction diagrams, setup instructions, API contracts, validation results and project handoff. It explains how Profile coordinates Fact and Profile Builder, preserves source evidence and user decisions, and supplies the current profile to Master.

The standalone project runs as a Node.js library and local command line tool. It preserves the existing source and prompts. Its offline checks passed 185 tests; live operation requires credentials configured locally.

Public repository

Original Career Scout repository

Contents

Project overview 4

Start here . . . 4

Run without credentials . . . 4

Agent structure . . . 5

Preservation and provenance . . . 5

Profile Agent design and implementation 6

1 Purpose and scope . . . 6

2 Responsibilities and boundaries . . . 6

3 Architecture diagram . . . 7

4 Resume and Fact Agent workflow . . . 8

5 Profile Builder workflow . . . 8

6 User decision and persistence sequence . . . 10

7 Current profile handoff . . . 11

8 Storage and evidence model . . . 11

9 Tools APIs and frameworks used by the implementation . . . 12

10 Prompt and implementation map . . . 13

11 Python graph and extraction boundary . . . 14

12 Guardrails and known limits . . . 14

Setup and operation 15

Requirements . . . 15

Install and verify offline behavior . . . 15

Configure live storage and models . . . 15

Live workflow . . . 16

Operational behavior 16

Troubleshooting 17

Integration and deployment limits 17

API and integration **18**

Trusted library interface 18

Current Profile actions 18

Example action files 19

Original Career Scout HTTP API 20

Provider boundaries 21

Validation record **21**

Executed checks 21

Scope and limitations 21

Project handoff **22**

Maintenance instructions **22**

Source files and provenance **24**

Preserved agent prompts 24

Project overview

Runnable Profile Agent, Fact Agent and Profile Builder extracted from Career Scout. Profile turns a resume and optional job descriptions into a source-backed, versioned profile that Master can use. It owns storage and user decisions; its two analysis agents never approve claims themselves.

This repository runs as a **Node.js library and local command-line tool**. It does not include Career Scout's browser interface or the downstream job-search/application service. Shared modules required by the original implementation remain included to preserve behavior.

Start here

Document	Covers
Profile Agent documentation	Purpose, responsibilities, diagrams, both subagents, evidence, memory, tools and APIs
Setup and operations	Installation, configuration, database setup, running the agent, troubleshooting
API and integration	Library methods, CLI commands, action contracts and Master handoff
Validation	Executed checks, preservation evidence and remaining limits
Upstream manifest	Source repository, exact source commit and hashes of unchanged files

Run without credentials

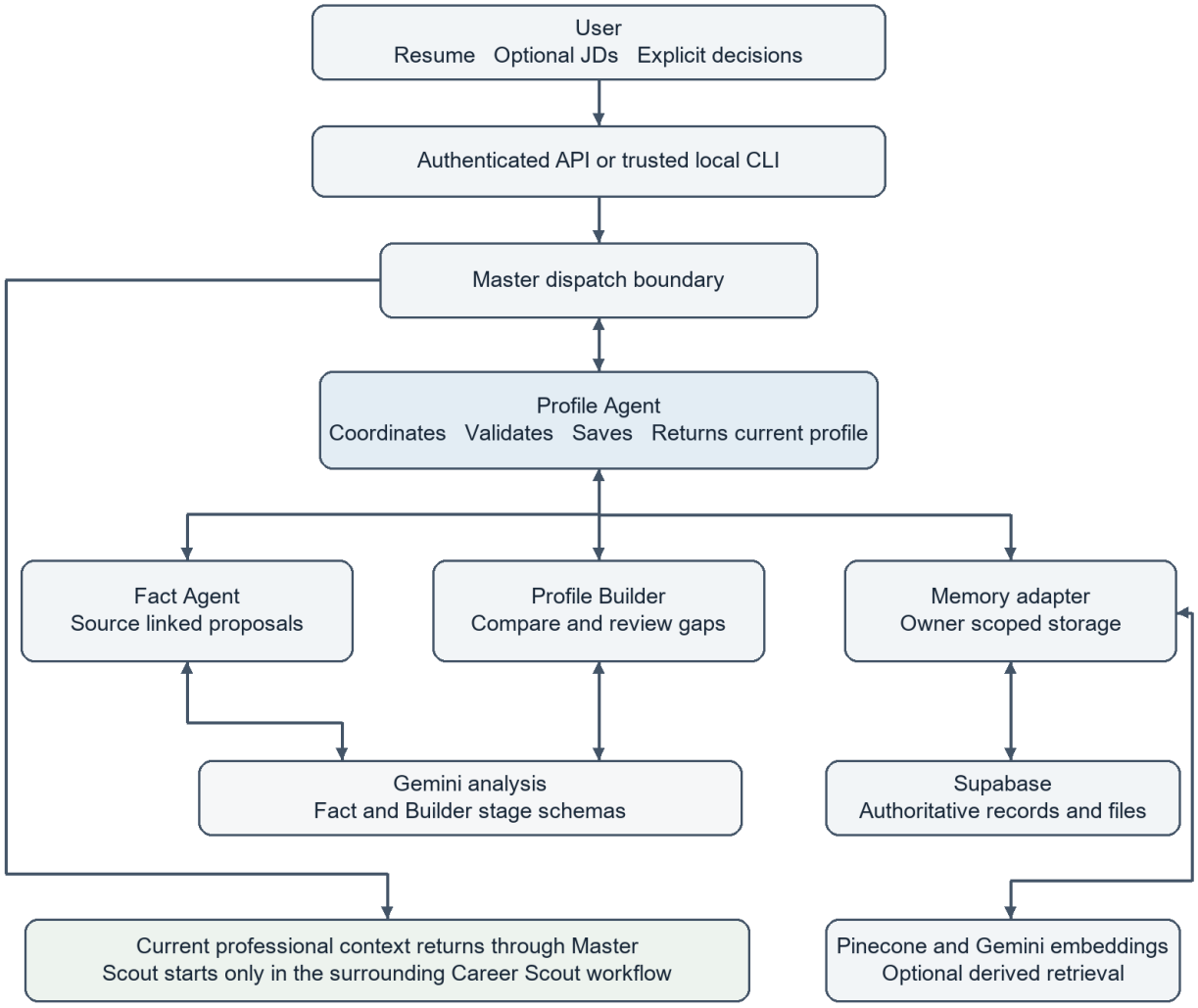
Install Node.js 22.13 or later, then:

```
npm ci
npm run demo
npm test
npm run verify:upstream
```

The demo uses synthetic facts, a simulated user approval and in-memory storage. It demonstrates verified saving and the current-profile handoff; it does not call Gemini, Supabase or Pinecone. Live operation setup is documented separately.

Agent structure

Figure 1 Agent coordination overview



Fact and Builder have no direct link to each other or to storage.

Memory supports the two analysis agents through Profile. It is not a third agent that writes experience. The standalone package returns the handoff to its caller; it does not start Scout.

Preservation and provenance

The 52 source, prompt, migration and regression-test files listed in `UPSTREAM.json` are copied unchanged from Career Scout commit `cedc267896578988c2c4607e5cd0877e45e28f39`. The standalone entry point, CLI, demo and documentation are new. Fact and Profile Builder prompts retain their original paths. Profile and Memory prompts are included as coordinator policy references; the entry point does not send them to a model or interpret them as executable routing.

No existing profile, resume, credentials or database export is included. The CLI requires a separate `profile-agent:` owner identity and does not load Career Scout's environment file. Do not configure a hosted public endpoint around this library without an authentication boundary.

There is no license grant added by this extraction. Public repository visibility alone does not grant a software license; upstream and dependency licensing must be respected.

Profile Agent design and implementation

Documented November 5, 2026, America/New_York. Baseline: Career Scout commit cedc267. This document describes inspected code and prompts, not a claim that every provider or deployed environment is currently healthy.

1 Purpose and scope

Profile Agent maintains one current factual profile. It takes resume information, coordinates extraction, compares optional employer job descriptions with recorded experience, saves explicit user-confirmed additions and returns eligible professional context to Master.

Its useful output is more than a resume summary: each eligible record retains its identity, version, source quotation, lifecycle and confirmation status. Downstream agents can distinguish what the resume states from what the user later confirms and what remains unknown.

Profile does not discover vacancies, apply to employers, send reminders or rewrite a resume. Career Scout's separate downstream agents handle their respective stages. There is no automatic 70% job handoff in this Profile package. Builder's 75% rule selects useful profile questions; it is not a job match threshold.

The current Career Scout interface has four steps: Resume, Facts, Builder, and Search & shortlisted jobs. The Jobs and Applications header tabs were removed. Profile owns the first three stages and supplies the context for the fourth. The standalone repository provides the same core service through a library and CLI, without reproducing the browser screens.

2 Responsibilities and boundaries

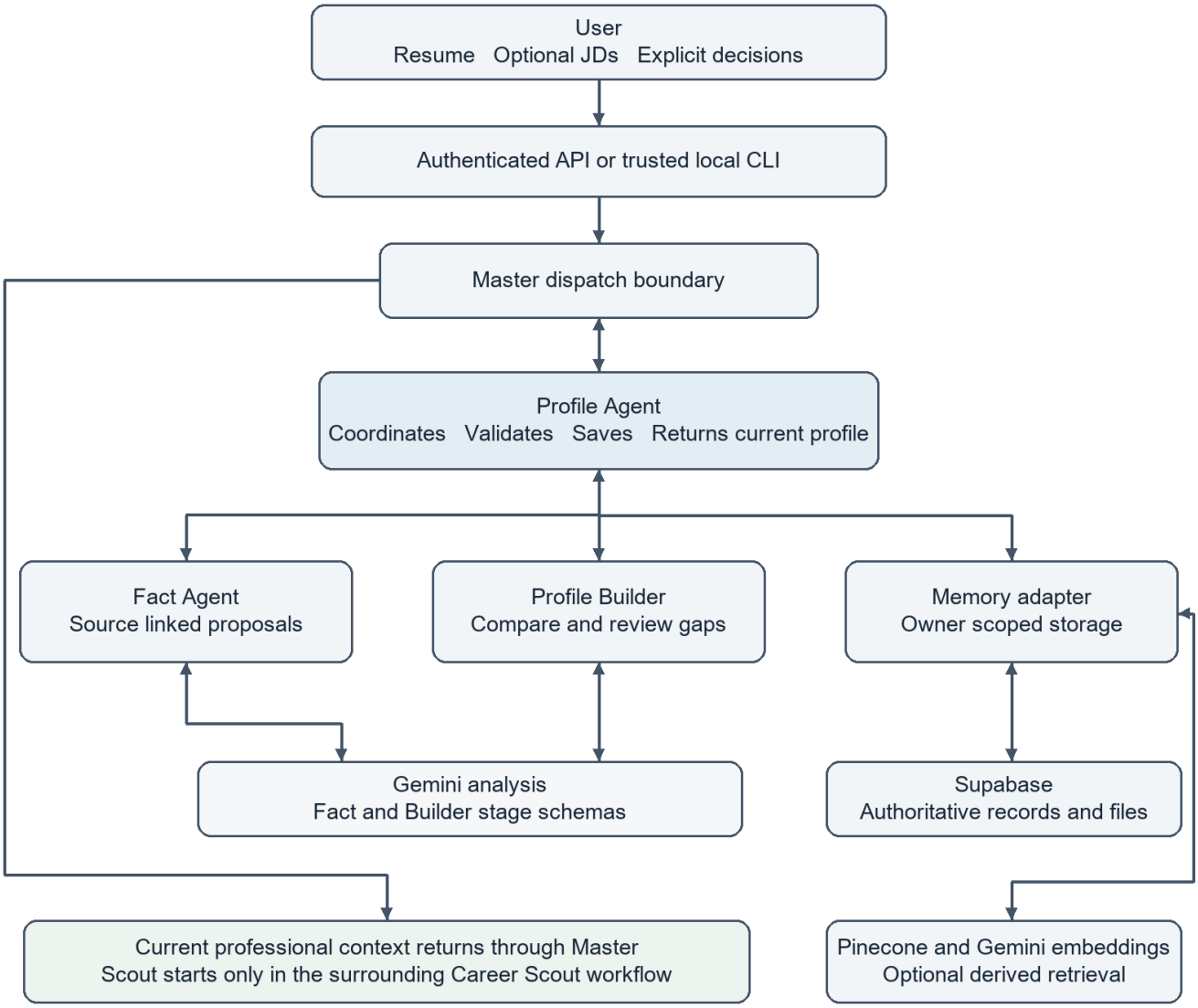
Component	Receives	Produces	Must not do
Master boundary	User operation or request for a profile	Scoped call to Profile; receives Profile result	Let other agents bypass Profile to read profile memory
Profile coordinator	Trusted owner, documents, actions, current revision	Stored decisions, verified save receipts, current-profile packet	Invent evidence, approve on behalf of the user, start research from a save
Fact Agent	Resume or substantive answer source segments; supplied factual context	Structured factual proposals with exact source references	Treat a JD as personal experience, invent missing fields, approve proposals
Profile Builder	Optional JD requirements, approved facts, source records and earlier decisions	Coverage assessments and focused experience/skill questions	Turn employer requirements into candidate facts or save its own answers

Component	Receives	Produces	Must not do
Memory adapter	Profile-owned load/save/retrieval operations	Authoritative records, files, derived retrieval results	Treat vector similarity as factual proof

Fact and Builder do not communicate directly. Profile supplies each stage with a scoped payload and receives its result. Only Profile accesses memory on their behalf. Master remains the top-level coordinator.

3 Architecture diagram

Figure 2 Profile service architecture



Fact and Builder have no direct link to each other or to storage.

The JavaScript service uses small LangGraph child-task graphs to enforce Profile → child → Profile routing. Coordination, approvals, storage, validation and content hashing are implemented in code. They are not delegated to a model choosing arbitrary tools.

4 Resume and Fact Agent workflow

1. The trusted caller supplies a resume file or pasted text and the expected storage revision.
2. Code checks document limits, extracts text and assigns stable document/segment identifiers. Supported files are text-based PDF, DOCX and UTF-8 TXT.
3. Profile saves the source and performs the existing Section 1 extraction stage. Full model analysis is a separate action; uploading alone must not be described as completed Gemini analysis.
4. Fact's editable prompt and stage-specific JSON schema guide Gemini over supplied resume/answer segments.
5. Code grounds returned proposals in surviving source quotations, validates records and preserves existing decisions. The extraction stage tracks referenced and unreviewed passages.
6. Profile stores the result. New model proposals are not automatically user-approved. Eligible source-matched resume records may nevertheless be read by the current-profile projection as `source_stated`.

Fact extracts personal details, distinct employment stints, education, certifications, listed skills and professional work assertions. Work assertions retain supported responsibilities, tools actually used, outcomes and scope. Merely listing a tool does not prove a project or years of experience using it.

Code calculates employment durations using supplied date precision and avoids double-counting overlapping periods. It must not infer an absent end date or resolve a two-digit year to an invented century. Derived tenure is not a new approved fact.

Limits in the copied implementation include 2,000,000 file bytes, 60,000 text characters, 800 segments, 80 active documents and at most 25 PDF pages. PDFs containing pages without extractable text are rejected; this implementation does not add OCR.

5 Profile Builder workflow

JDs are optional for the factual profile. When supplied, each is retained as employer context. A JD may be uploaded, pasted or read from a supported public link; a title alone cannot establish its requirements. URL intake filters unsafe destinations and removes unrelated page content where it can identify it.

Profile Builder has two separate evidence channels:

Channel	What it means
Approved facts	IDs of supplied facts with valid approval; support only their exact confirmed scope
Recorded source coverage	Resume/source records, including eligible pending resume statements; prevents asking the same experience question just because approval is pending

The service extracts JD requirements, then compares them in batches of up to eight. Proposed questions and invalid comparisons receive a separate evidence review in batches of at most two

requirements. The second pass receives source candidates and validation issues without the previous score. This is another bounded Builder stage, **not a separate Reviewer Agent** and not Career Scout's downstream Critic.

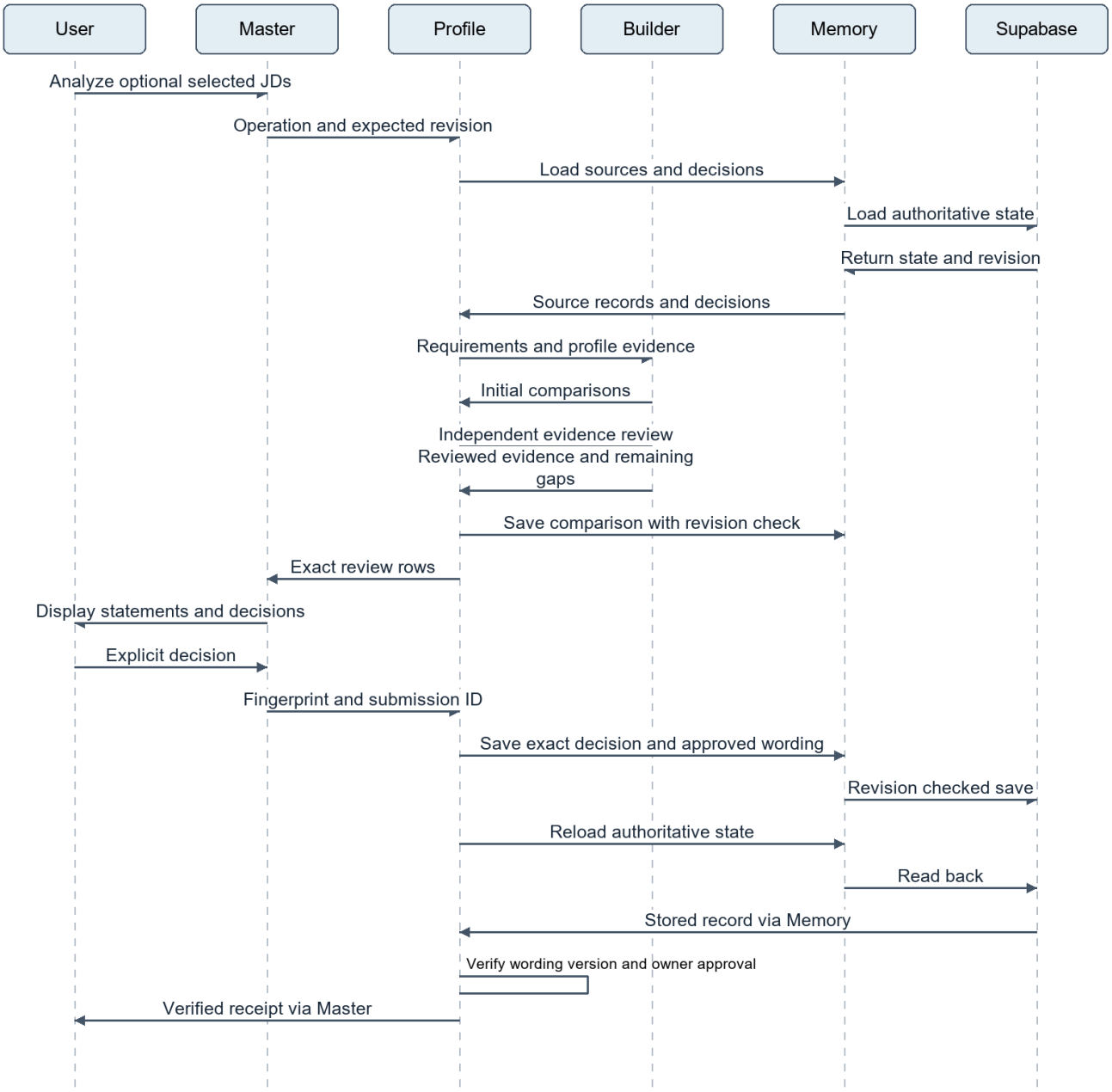
Code validates component references and calculates weighted coverage: matched weight / total weight × 100. Core components weigh 3 and minor details weigh 1. At 75% or above, a question is skipped only if remaining gaps are minor and no core or critical gap remains. At 50–74%, questions focus on missing material scope. Below 50%, material or critical gaps can be asked; incidental differences may be deferred. Required qualifications are routed to factual review, not invented as work experience.

Questions are grouped into separate Professional Work Experience and Skills tables for each role. Previous decisions are reused. Unresolved comparisons remain explicit and do not become broad approvable claims. A skipped detail remains unconfirmed.

Current code limitation: `buildProfiles` checks for at least one approved professional fact before comparison. This is distinct from current-profile search readiness, which can use eligible resume information without repeat approval. The extracted package preserves that check and its existing error message; documentation must not promise a completely approval-free Builder comparison.

6 User decision and persistence sequence

Figure 3 Builder review and verified saving



Approve adds only the exact confirmed wording and requires read back verification.
Reject records the decision and adds no claim.
Somewhat saves a draft and requires separate approval of revised wording.
No save starts job search. Retries retain the same submission identity.

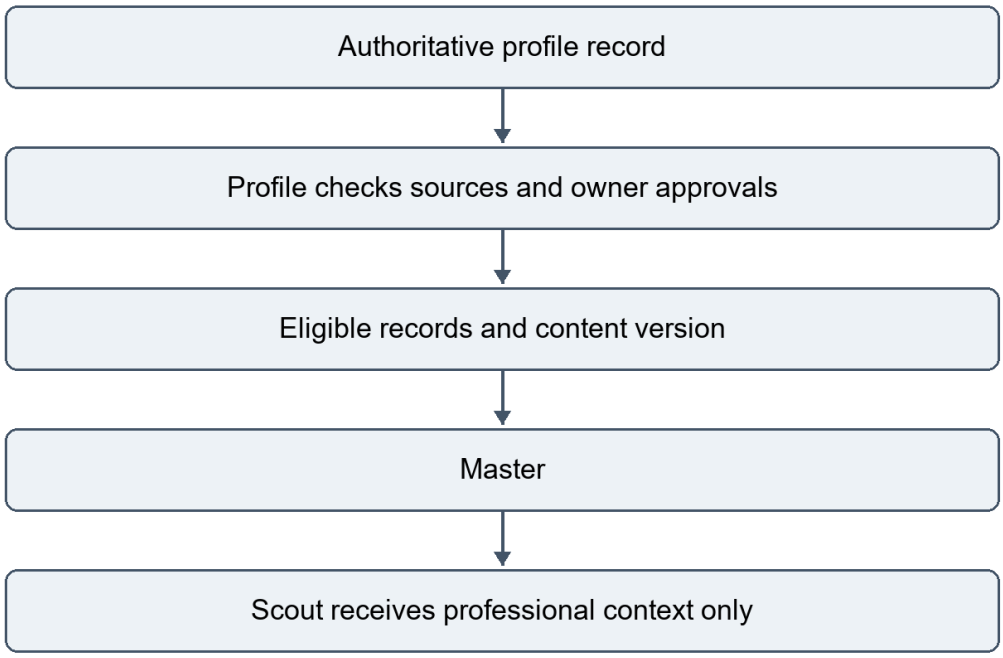
Retries use the same submission ID and payload. Reusing an ID with different decisions is rejected. Row fingerprints and revisions prevent stale approvals. Read-back verification is required before reporting that an approved addition was saved successfully. No extra Save Final Profile action is required by the current workflow.

7 Current profile handoff

`currentProfilePacket` loads existing authoritative records and builds a derived view; it does not create another stored profile or approve records. Eligible information consists of valid owner-approved records, eligible source-matched pending resume records and exact resume work passages supported by the factual view. Rejected, withdrawn, conflicting, missing-source and invalid-approval records are excluded. Unapproved Builder drafts are excluded.

The content version has the form `current-<SHA256>`. Its hash binds the owner, eligible records, professional search profile, search preferences and disclosure preferences. A pinned request fails if this content changes. The packet includes facts, search profile, preferences, source references and a verification receipt.

Figure 4 Current profile handoff



The full packet can contain personal information and belongs at the trusted coordinator boundary. Scout receives only the professional search context, not raw resumes, contact details or storage credentials. Reading or saving Profile does not start job research.

8 Storage and evidence model

Data	Authoritative location / treatment
Original uploaded bytes	Supabase private bucket `career-profile-private`, under a hashed owner path
Profile state and revision	Supabase `career_profile_states`; revision-checked RPC saves
Usage reservations	Supabase `career_profile_usage`; bounded operation reservations
Documents and segments	Profile state with IDs, kind, source text, lifecycle and extraction metadata

Data	Authoritative location / treatment
Facts and decisions	Versioned records, evidence quotes, approval references and decision history
JD requirements	Employer-only context with separate source evidence
Builder reviews/drafts	Profile state; only explicit valid approval adds a confirmed claim
Semantic vectors	Derived Pinecone namespace; revalidated against authoritative records
Current profile	Computed from current eligible content; legacy snapshots remain compatibility data

`source_stated`, `user_confirmed` and `independently_verified` have different meanings. A matching quote proves source correspondence, not external truth. User approval confirms wording; it is not independent employment or credential verification. Vector similarity only retrieves candidates for checking.

9 Tools APIs and frameworks used by the implementation

These names and model IDs are read from the pinned code; they are not a current provider-availability or pricing claim.

Tool / API	Use and implementation
Node.js 22.13+ / JavaScript modules	Runs the standalone coordinator, CLI and tests
`@langchain/langgraph` 1.5.2	Scoped Profile → Fact/Builder → Profile child execution in `service.mjs`
Gemini `gemini-3.1-flash-lite`	Structured Fact extraction, JD extraction and Builder assessments via `/v1beta/models/{model}:generateContent`
Gemini `gemini-embedding-001`	Optional 3072-dimensional retrieval vectors via `:batchEmbedContents`
Supabase REST RPC	`career_profile_load`, `save`, `reserve`, `purge`, `health`; server-only credentials and owner-scoped arguments
Supabase Storage API	Upload, download and delete private original documents
Pinecone data API	`/vectors/upsert`, `/vectors/delete`, `/query`; optional derived retrieval index
`unpdf`	Extract text from text-based PDFs; original input bytes preserved
`fflate` + `fast-xml-parser`	Bounded DOCX archive/XML extraction
Native Fetch and Web Crypto	Provider requests, bounded network operations, fingerprints and hashes
`node:test` + PGlite	Offline regressions and PostgreSQL migration behavior tests
React 19 / TypeScript / Vinext	Career Scout's original browser/API host; not required by this standalone library

Keenable performs downstream Scout discovery in Career Scout. It is not the model provider for Fact or Builder. No Keenable key is needed for this standalone entry point. Shared Scout-related modules remain in the import closure for existing normalization, JD handling and compatibility; no discovery command is exposed.

10 Prompt and implementation map

File	Responsibility
`lib/profile/service.mjs`	Current web-runtime coordinator, child tasks, analysis, saves and loaders
`lib/workflow/profile-intake.mjs`	Master → Profile intake audit boundary
`lib/workflow/policy.mjs`	Allowed communication routes
`backend/src/career_scout/agents/fact/prompt.md`	Fact extraction/evidence policy
`backend/src/career_scout/agents/profile_builder/prompt.md`	Comparison, independent review, question and decision policy
`backend/src/career_scout/agents/profile/prompt.md`	Coordinator responsibility reference
`backend/src/career_scout/agents/memory/prompt.md`	Memory responsibility reference
`lib/profile/providers.mjs`	Gemini, Supabase and Pinecone adapters and stage schemas
`lib/profile/current-profile.mjs`	Current eligible records and content-bound packet
`lib/profile/builder-expansion.mjs`	Review rows, exact decisions, replay and approval checks
`lib/profile/source-records.mjs`	Valid source and confirmation eligibility
`lib/profile/builder-evidence-validation.mjs`	Candidate accounting and evidence consistency
`src/index.mjs`	New standalone trusted-library entry point; fixed prompt loading
`bin/profile-agent.mjs`	New local CLI with isolated owner identity

In the original web app, Fact/Builder prompts load from the local Python prompt endpoint when configured, otherwise from build-time bundled Markdown. The standalone library loads the unchanged

files from disk for each model stage. Runtime JSON schemas still control outputs; prompt edits do not create permissions or bypass code checks.

11 Python graph and extraction boundary

Career Scout also has a Python LangGraph Profile graph and typed Fact/Builder tools. Their tool names are `extract_facts` and `build_profile`. The inspected Python coordinator retains legacy `needs_fact_approval` / `needs_profile_save` states, and its coordinator currently selects Fact or Memory; the Builder node exists but is not selected by that coordinator path. It is not the active implementation of the current browser questionnaire workflow.

This repository therefore extracts the working JavaScript Profile service. It preserves the shared Markdown prompt paths but does not ship an incomplete Python graph as a second runnable Profile system. The `pythonProfileLoader` compatibility conversion remains in the copied JavaScript service.

12 Guardrails and known limits

- Owner identity comes from the trusted caller. The original HTTP host authenticates users and enforces same-origin and bounded requests; this local library is not a replacement HTTP authentication service.
- Documents and tool output are untrusted evidence. Exact references and code checks constrain model output; a prompt alone cannot guarantee perfect extraction or defeat every malicious input.
- Storage updates check revisions. Idempotent review operations preserve retries and decisions. Source withdrawal invalidates dependent evidence.
- Model/provider failures remain failures. Truncated or structurally invalid outputs are rejected; bounded evidence review does not authorize infinite retries.
- Semantic search is optional; database retrieval remains available when embeddings or Pinecone are disabled/unavailable.
- Historical Fact comparison found omitted fields/skills and duplicate-record issues. A referenced passage or zero unreviewed passages does not prove complete ingestion. The current package does not claim those quality issues are solved.
- Legacy snapshot/finalization functions remain for compatibility, including older error wording. The standalone public action list omits the retired generated-role and final-save paths; the current-profile loader supplies the current handoff.
- Offline tests establish the tested behavior only. Live provider credentials, model availability, representative resume quality, interactive browser acceptance and hosted deployment are separate checks.

See setup, API and validation for the executable interface and verification scope.

Setup and operation

Requirements

- Node.js 22.13 or later and npm.
- Supabase project with the supplied Profile migrations for durable live storage.
- Gemini API credentials for live Fact/Builder model calls. No OpenAI or Groq integration is used by this package.
- Optional Pinecone index compatible with 3072-dimensional Gemini vectors for semantic retrieval.

The checked-in model IDs are inherited from Career Scout. Provider availability and quotas must be verified in the environment where you run the package. There is no automatic alternate-provider fallback.

Install and verify offline behavior

```
git clone https://github.com/ujjwalbhateja/Career-Scout-Profile-Agent.git
cd Career-Scout-Profile-Agent
npm ci
npm run demo
npm test
npm run verify:upstream
```

The demo and test suite use synthetic fixtures and do not require credentials. They do not establish live model quality. `verify:upstream` checks the preserved source, prompts, migrations and tests against their recorded hashes.

Configure live storage and models

Copy `.env.example` to ignored `.env.local`. Fill in values locally, never in documentation or Git:

Variable	Use
`PROFILE_OWNER_ID`	CLI-only trusted local identity; choose a stable unique `profile-agent:` value, replacing the example
`SUPABASE_URL`	Project URL for the authoritative store
`SUPABASE_SECRET_KEY`	Server-only Supabase secret; adapter also accepts legacy `SUPABASE_SERVICE_ROLE_KEY`
`GEMINI_API_KEY`	Fact/Builder analysis and optional embeddings
`PINECONE_API_KEY`	Optional semantic index access
`PINECONE_INDEX_HOST`	Optional data-plane index host
`PROFILE_SEMANTIC_SEARCH`	`disabled` initially; remove/set otherwise when intentionally enabling semantic calls

The standalone process does not read Career Scout's `.env.local`, copy its credentials or infer its authenticated owner. The CLI prefix prevents accidental reuse of its ordinary owner identity. Reusing a Supabase project is possible after configuration, but the new owner must remain distinct if separate profile data is intended. No credentials or live data were configured as part of this extraction.

For a new database, apply these files in order using a project administrator's Supabase SQL tooling:

1. `supabase/migrations/202610280001_profile_memory.sql`
2. `supabase/migrations/202610290002_profile_conflict_response.sql`

The migrations create the Profile state/usage tables, guarded RPCs and a private document bucket. They are not run automatically by the CLI or `npm ci`. Existing installations should be inspected before migrations are applied. The standalone task has not modified Career Scout's database schema.

Live workflow

1. Run `npm start -- status`. This reports configuration presence, not provider authentication or model health.
2. Run `npm start -- read` to load the configured owner's record and current revision. For a new owner this should be empty after the database is prepared.
3. Upload with `npm start -- upload "C:\private\resume.pdf" 0 resume`, substituting the actual latest revision. Or use a private JSON `paste` action. Intake performs the existing deterministic Section 1 parsing; it does not establish completed Gemini analysis.
4. Inspect the returned documents and proposals. Use explicit `analyze` for the saved resume document when full Fact model extraction is wanted. This calls Gemini and may consume quota.
5. Optional Builder path: inspect and explicitly decide professional facts, then upload/paste/import and select JDs. Run `buildProfiles` with `mode: "expansion"`. Current code requires at least one approved professional fact. No automatic approval is supplied by this CLI.
6. Run `npm start -- questionnaires`. Inspect the exact visible statements and their fingerprints. Submit `reviewBuilder` decisions only for what the user actually confirms. Somewhat saves a draft; approving revised wording is a separate action.
7. Run `npm start -- current` to obtain the source-backed current profile for Master. No separate final-save action or automatic job search occurs.

Action files must use the latest revision, IDs and fingerprints returned by the service. Examples in API.md are templates, not real user decisions. Keep action files, resumes and saved outputs under ignored `.local/` or outside the repository. CLI JSON output can contain private profile information.

Operational behavior

Supabase is authoritative. Revision conflicts require a reload, not a forced overwrite. If an approval response is lost, retry its same submission ID and identical payload; the service checks replay and verifies storage. Changing a payload requires a new submission ID and fresh row basis.

The original API can schedule indexing after certain actions. This CLI does not install a background scheduler. Request `sync` explicitly after enabling and configuring semantic search. Database retrieval can operate with semantic search disabled.

Removing a document withdraws it and its dependent evidence from active use; original file cleanup is separate. `deleteProfile` requires the exact confirmation phrase and can remain pending if configured derived stores cannot be purged. These actions must be explicit.

Troubleshooting

Symptom	Meaning / next step
Missing trusted owner	Set a stable unique `profile-agent:` identity; the CLI does not authenticate remote users
Supabase not configured	Set the URL and server-only secret; do not paste them into commands or commits
Schema/RPC error	Check that the Profile migrations are installed in the selected project
Stale revision / profile version	Read current state, inspect changes and resubmit against the new basis
Builder asks for approved experience	Existing comparison prerequisite; inspect and explicitly approve valid professional facts
Gemini unavailable / quota exhausted	Preserve saved input; report the failure and retry deliberately when available
Could not confirm saved decision	Reload; retry the identical submission if appropriate. Do not assume the failed response means no write occurred
No extractable PDF text	Supply a complete text-based PDF, DOCX or TXT; OCR is not included
Comparison unresolved	Retain uncertainty; do not convert it into an invented approval question
Semantic retrieval unavailable	Database retrieval remains available; inspect optional configuration and index compatibility

Integration and deployment limits

The library accepts a trusted `owner` argument from server code. Do not accept it unvalidated from a browser body. Any future HTTP host needs authenticated owner resolution, request-size limits, same-origin/CSRF handling where applicable, private responses and protection for server credentials. The original Career Scout host provides those boundaries; this package does not ship a new web server.

The Python service, React UI, Scout discovery, matching, tailoring, Critic, applications and reminders are outside this standalone entry point. Use Career Scout for that surrounding application. Shared imported modules are preserved for compatibility; their presence does not mean those workflows are exposed or started here.

API and integration

Trusted library interface

```
import {createProfileAgent} from './src/index.mjs';

// Resolve owner from trusted server authentication, or explicit local configuration.
const profile = createProfileAgent({env: process.env, owner: trustedOwnerId});
const record = await profile.read();
const packet = await profile.current();
```

The `owner` argument is mandatory. The wrapper filters environment keys, rejects owner fields in action payloads and loads prompts from fixed paths. This is a local/server API; it does not authenticate an arbitrary caller on its own.

Method	Result / behavior
`status()`	Configuration flags and code-selected model names; no network check
`read()`	Latest `{revision, state, currentProfile, finalProfileStatus, providers, limits}`; legacy final-status metadata is not a new workflow gate
`current(version = 'current')`	Current eligible packet; pinned `current-…` fails on changed content
`questionnaires()`	`{revision, roles}` containing recorded capabilities, unresolved/skipped gaps, experience and skill review rows
`upload(file, {kind, title, selected, revision})`	File intake through the Master/Profile dispatch boundary; stored source and deterministic Section 1 parsing for resumes
`execute(body)`	Allowlisted Profile operation through the Master/Profile boundary; no client-controlled ownership
`readPrompt(name)` export	Fixed `fact`, `builder`, `profile` or `memory` policy file; no caller-supplied path

The current packet contains `facts`, `roleProfiles` (empty for the current workflow), `searchProfile`, `snapshot` (a derived compatibility descriptor), `profileSummary.saveVerification`, `searchPreferences` and `searchHistory`. A `snapshot` field here does not mean a new snapshot was persisted. `source_validated` means source matching, not independent verification.

Current Profile actions

Every `execute` action includes the current numeric `revision`. Builder replay recognizes an identical prior submission before rejecting an old revision; do not rely on this for changed payloads.

Action	Additional fields / purpose
`paste`	`kind: resume|jd`, `text`, optional `title`, `selected`; save source
`importJD`	`url`; retrieve a readable public JD safely

Action	Additional fields / purpose
`selectJD`	`id`, `selected`; select optional JD context
`extractSection1`	`id` of resume; existing deterministic structured extraction
`analyze`	`id`; Fact model extraction for resume/answer, Builder extraction for a JD
`prepare`	Analyze pending resume/answer and selected JD documents; optional `search` preferences
`buildProfiles`	`mode: expansion`, optional `documentIds`; compare selected requirements and prepare review rows
`decideFacts`	`decisions: [{id, version, decision}]`; explicit per-version factual decisions
`reviewBuilder`	`submissionId`, `rows`; exact questionnaire decisions described below
`correctFact`	Existing domain correction contract in `lib/profile/domain.mjs`; creates a new version requiring its own decision
`resolveConflict`	`id`; explicit choice of retained factual record under existing domain rules
`answer` / `analyzeAnswer`	Compatibility questionnaire answer path; see `addAnswer` in `domain.mjs`; current role tables use `reviewBuilder`
`preferences`	`search` and/or `disclosure`; saves explicit preferences
`retrieve`	`query`, optional `kind`; authoritative database/optional semantic candidates
`sync`	Explicit derived-index work; requires configured/enabled semantic providers
`removeDocument`	`id`; withdraw source and invalidate dependent active use
`cleanup`	Complete queued original-file deletion
`export`	Profile JSON and original-file metadata; the inherited download paths refer to Career Scout's HTTP route, not a new CLI server
`deleteProfile`	`confirmation: "DELETE MY PROFILE"`; explicitly delete the configured owner's profile

Retired generated-role approval and legacy `finalize`/`saveFinalProfile` are not exposed by this standalone wrapper. Their copied implementation remains available internally for compatibility tests; the recommended handoff is `current()`.

Example action files

Save private action files under ignored `.local/`, then run:

```
npm start -- action .local/action.json
```

Full resume analysis, using the ID and revision returned by intake:

```
{"action":"analyze","revision":1,"id":"REPLACE_WITH_SAVED_RESUME_ID"}
```

Analyze selected optional roles and prepare questions:

```
{"action":"buildProfiles","revision":4,"mode":"expansion"}
```

Explicit approval of one displayed Builder row:

```
{
  "action":"reviewBuilder",
  "revision":5,
  "submissionId":"REPLACE_WITH_UNIQUE_SUBMISSION_ID",
  "rows":[{
    "rowId":"REPLACE_WITH_RETURNED_ROW_ID",
    "fingerprint":"COPY_THE_EXACT_RETURNED_FINGERPRINT",
    "decision":"approve"
  }]
}
```

Allowed Builder decisions are `approve`, `reject`, `somewhat` and `approve_revision`. `somewhat` requires the user's precise `text` and stores a draft. Refresh the questionnaires before submitting `approve_revision`, supplying the exact revised `text`, current `draftVersion` and fingerprint returned for that row. Do not invent a row ID, fingerprint, approval receipt or statement.

A successful `reviewBuilder` result includes `kind: builder_review_saved`, the submission ID, decisions/fact references, `storageVerified: true` and a current-profile summary. If read-back fails, the operation reports failure to confirm; an earlier database write may already have succeeded, so replay the same submission deliberately instead of generating a duplicate approval.

Original Career Scout HTTP API

The source application exposes these routes; this standalone repository does not start them:

Route	Behavior
`GET /api/profile`	Authenticated owner-scoped profile read
`GET /api/profile?status`	Configuration flags
`GET /api/profile?download=<documentId>`	Authorized private original download
`POST /api/profile` JSON	Same-origin bounded action dispatch
`POST /api/profile` multipart	Same-origin bounded file upload

The original API rejects supplied `owner`, `owner_id` or `namespace` fields, derives identity from its trusted host, and marks responses private/no-store. Host authentication must prevent clients forging trusted identity headers. Copying the library into a public server without that boundary is not equivalent to Career Scout's deployment.

Provider boundaries

Gemini receives scoped source material and an active stage schema. Profile prompts do not grant network access or storage rights to model output. Supabase is contacted by the Profile memory adapter with server credentials. Pinecone results are checked against surviving authoritative records. Unknown or failed provider responses do not authorize fabricated facts, automatic approvals or a claim of successful saving.

The CLI returns JSON to stdout and errors to stderr with a nonzero exit code. Library operations preserve `ProfileError` status where raised by upstream code, including stale revision/version and unavailable-provider conditions. Do not expose raw profile outputs in public logs.

Validation record

Date: November 5, 2026, America/New_York.

Executed checks

- Independent dependency installation succeeded in this standalone project.
- **185 tests passed**, zero failures: copied factual grounding, current-profile, Builder, source coverage, question policy, provider behavior, PostgreSQL migration and related regression checks, plus standalone ownership/action-boundary and synthetic-demo checks.
- The synthetic demo exercised explicit simulated Builder approval, revision-checked in-memory saving, read-back verification and the current-profile handoff. Its test forbids network calls.
- All **52 upstream files** matched their recorded SHA256 hashes from Career Scout commit `cedc267896578988c2c4607e5cd0877e45e28f39`, including the four preserved Profile-family prompts.

Scope and limitations

The tests use synthetic data and mocked providers or local PGlite. No live Gemini request, Supabase profile write, Pinecone indexing or real resume analysis was performed for this extraction. Credentials were not copied, migrations were not applied to an existing database and the original Career Scout application code was not changed.

The package is runnable locally as a library/CLI. It is not a new browser app or hosted service. The synthetic demo is not evidence of live model extraction quality, complete resume ingestion or production deployment. Provider availability, authentication and representative end-to-end model results remain configuration-dependent checks.

Historical ingestion omissions, duplicate grouping risks, the Builder approved-fact prerequisite and legacy snapshot messages are documented in PROFILE_AGENT.md. They are preserved rather than silently changed during extraction.

Project handoff

Updated November 5, 2026, America/New_York.

The user requested Profile Agent documentation, diagrams, an account of its two analysis agents and tools/APIs, and a separate public GitHub repository containing runnable code and documentation. Repository name: `ujjwalbhateja/Career-Scout-Profile-Agent`.

The active Profile workflow is implemented in Career Scout's JavaScript service. This project extracts that service and its required dependency closure unchanged from commit `cedc267896578988c2c4607e5cd0877e45e28f39`. The legacy Python Profile graph is documented but is not shipped as a second runnable implementation. The original prompt paths are preserved; all four Profile-family Markdown policies are unchanged.

The new trusted-library entry point, CLI and offline demo provide independent execution. This is not a browser interface. Live setup requires explicit credentials and a distinct CLI owner beginning `profile-agent:`. No existing Career Scout secrets, profiles, resumes or databases were copied or changed. No live provider call or database migration was performed for extraction.

Validation: 185 tests passed, standalone CLI help/demo ran successfully, local Markdown links and fences checked, and 52 upstream file hashes matched. The demo uses synthetic model outputs and a simulated user decision; it is not live Fact/Builder model validation. See `VALIDATION.md` and `UPSTREAM.json`.

Keep the original extraction limits visible: Builder currently requires approved professional factual context; current-profile search readiness can use eligible resume statements. Preserve known ingestion quality limits, current versus legacy snapshot distinctions, explicit approval, source evidence, revision checks, idempotent replay and read-back verification.

The user authorized public publication. Git history and the remote provide the publication receipt. Future changes to copied source must record their divergence rather than silently preserving a false upstream-equivalence claim.

Maintenance instructions

- Keep this repository scoped to Profile, Fact, Profile Builder and Profile-owned memory. Master is the external coordinator boundary; never let the analysis children access each other's state or memory directly.
- Read `docs/PROFILE_AGENT.md` and `docs/SETUP.md` before changing behavior. Preserve exact evidence, owner scoping, revisions, explicit decisions, idempotent retries and verified save receipts.
- Do not alter prompts as part of packaging or routine documentation work. The copied prompt files and source hashes are recorded in `UPSTREAM.json`.
- `UPSTREAM.json` records the extraction baseline. If an upstream file is deliberately changed later, document the divergence and update the preservation strategy; do not silently claim byte-identical upstream behavior.

- Run `npm test` for behavioral changes and `npm run verify:upstream` for preservation checks. Record actual validation in `docs/VALIDATION.md`; fixtures are not live-provider evidence.
- Keep `.env.local`, credentials, real resumes, private outputs and local caches out of Git. Use `.local/` for private operation files and fixtures containing personal data.
- Never use the existing Career Scout owner/profile for tests. The standalone CLI uses an explicit separate `profile-agent:` identity; library integrations must resolve owner identity from trusted server context.
- This repository does not include a browser server or downstream Scout/applications. Do not present copied shared utility modules as newly activated workflows.

Source files and provenance

This manual consolidates the seven documentation files listed below. The repository preserves the original source, editable prompt files and upstream hash manifest. Source references are clickable links.

Project overview

Profile Agent design and implementation

Setup and operation

API and integration

Validation record

Project handoff

Maintenance instructions

Upstream source and SHA256 manifest

Preserved agent prompts

Profile prompt

Fact prompt

Profile Builder prompt

Memory prompt